Hardware Batch Submission Strategies for Intel DSA

Transparent Batching in the dsa-stdexec Framework

1 Overview

DSA processes work by receiving 64-byte descriptors through MMIO. Each doorbell write (`MOVDIR64B/ENQCMD`) is expensive — it triggers a cache-coherency transaction to the device. The DSA *batch opcode* (`0x01`) lets software submit a pointer to a contiguous descriptor array with a single doorbell, amortizing this cost.

Strategy	**Submitter class**	**Doorbells/N**	**In-flight**	**Blocks?**
Key idea				
Immediate	`DirectSubmitter`	N	N/A	No
1 doorbell per descriptor	Double-Buffered	`StagingSubmitter`	$\lceil N/B \rceil$	2
Yes	Two fixed arrays, swap on submit	Fixed-Ring	`FixedRingSubmitter`	$\lceil N/B \rceil$
16	No	Ring of fixed-size batch entries	Ring-Buffer	`RingSubmitter`
$\lceil N/B \rceil$	16	No	Shared descriptor ring + batch metadata ring	Mirrored-Ring
`MirroredRingSubmitter`	$\lceil N/B \rceil$	16	No	VM-aliased ring — no wrap-around logic

Table 1: Strategy overview. N = operations, B = max batch size (typically 32). All submitters satisfy the `DescriptorSubmitter` concept.

Key result: Ring-buffer and mirrored-ring batching achieve **1.2–2.0x** higher message rates than double-buffered batching across all 8 DSA operations, by eliminating submission blocking and improving batch utilization. Fixed-ring batching (ablation study) confirms that most of the gain comes from deeper in-flight capacity (16 vs. 2 batches), not memory

packing. The mirrored-ring variant further simplifies the implementation by eliminating wrap-around handling via virtual memory mirroring.

All five strategies present the same interface through `DsaProxy` type erasure (via Microsoft's proxy library) — scheduling strategies work identically across all backends. Each submitter is composed with `DsaEngine<Submitter, QueueTemplate>` to produce concrete DSA types (e.g. `DsaBatch = DsaEngine<StagingSubmitter, MutexTaskQueue>`).

2 How Batching Works

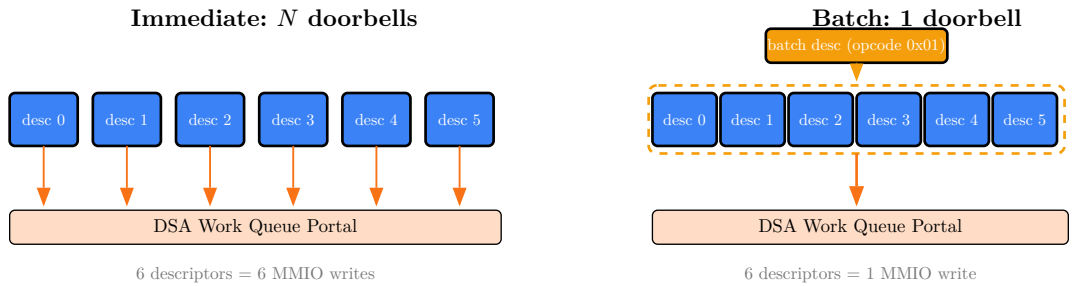

Figure 1: Immediate submission writes each descriptor individually. Batch submission sends one batch descriptor pointing to a contiguous array.

Hardware constraints for the batch opcode:

- Descriptor array must be **contiguous** and **64-byte aligned**
- Array must remain valid until hardware finishes DMA-reading it
- Minimum 2 descriptors per batch (unless device supports `Batch1`)
- Max batch size is per-WQ configurable (`accfg_wq_get_max_batch_size`)

3 Submission Strategies

3.1 Immediate (`DirectSubmitter`)

Idea: Submit each descriptor immediately via doorbell. N operations = N MMIO writes. Best for large transfers where doorbell cost is amortized by DMA time.

- Lowest per-operation latency (no staging)
- Highest doorbell cost

3.2 Double-Buffered (`StagingSubmitter`)

Idea: Stage descriptors into one of two fixed-size arrays. When full, submit as a hardware batch. Double-buffering allows filling one buffer while hardware reads the other.

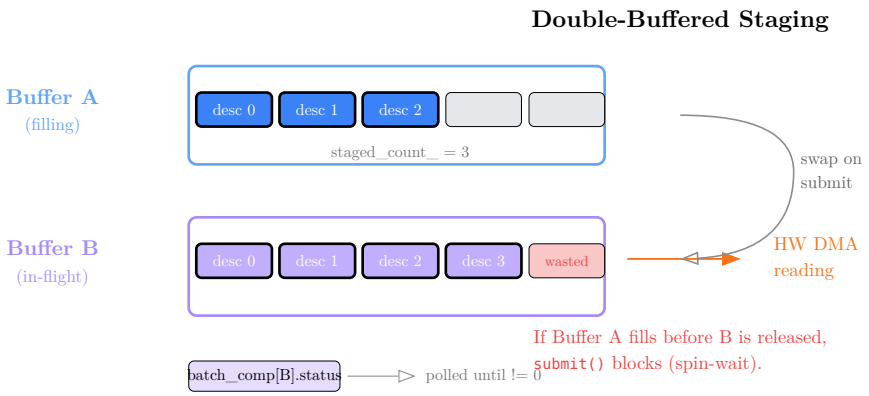

Figure 2: Double-buffered batching. Buffer A fills while hardware reads Buffer B. Fixed-size arrays waste unused slots. Submission blocks if both buffers are in use.

Limitations:

- **Submission blocks** on previous batch (only 2 buffers)
- **Effective batch size driven by poll frequency** – at low concurrency, most batches are size 1–4
- **Fixed-size buffers** waste space (32 slots reserved regardless of usage)

3.3 Fixed-Ring (`FixedRingSubmitter`)

> **Idea:** A ring of 16 fixed-size batch entries, each owning a contiguous 32-slot descriptor array plus a batch completion record. Eliminates the double-buffered blocking problem (16 slots vs. 2) without the wrap-around complexity of a shared descriptor ring.

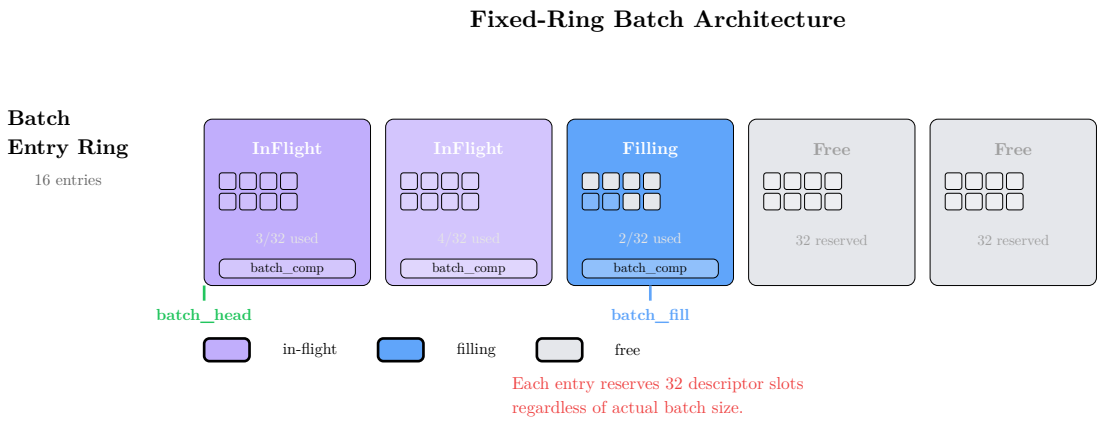

Figure 3: Fixed-ring architecture. Each batch entry owns a private 32-slot descriptor array. Unused slots (grey) are wasted when batches are small.

How it works:

- `submit()` copies the descriptor into the current Filling entry's private array. When `count >= max_batch_size_`, the entry is sealed and submitted as a hardware batch.
- `poll()` seals any partial Filling entry, then reclaims completed InFlight entries in order.
- No wrap-around logic needed — each entry's descriptor array is independent.

Trade-offs:

- **No blocking** (same as ring-buffer) – 16 batch slots provide sufficient depth
- **Simpler state management** – only `batch_fill_` and `batch_head_`, no `desc_head_`/`desc_tail_`

- **Wastes descriptor space** – each entry reserves B slots (32×64 B $= 2$ KB) even for small batches; total 16×2 KB $= 32$ KB
- **No contiguity requirement** across batches – each entry is self-contained

This strategy serves as an ablation baseline: it isolates the benefit of having 16 in-flight batches (vs. double-buffered's 2) from the ring-buffer's tighter memory packing.

3.4 Ring-Buffer (`RingSubmitter`)

> **Idea:** Two separate ring buffers — a descriptor ring (512 slots at default sizing, dynamically computed as `round_up_pow2(max_batch_size_ * num_batches_)`) for tightly-packed storage and a batch metadata ring (16 entries) for tracking in-flight batches. Full batches auto-submit; partial batches drain on `poll()`. 16 in-flight batches eliminate blocking.

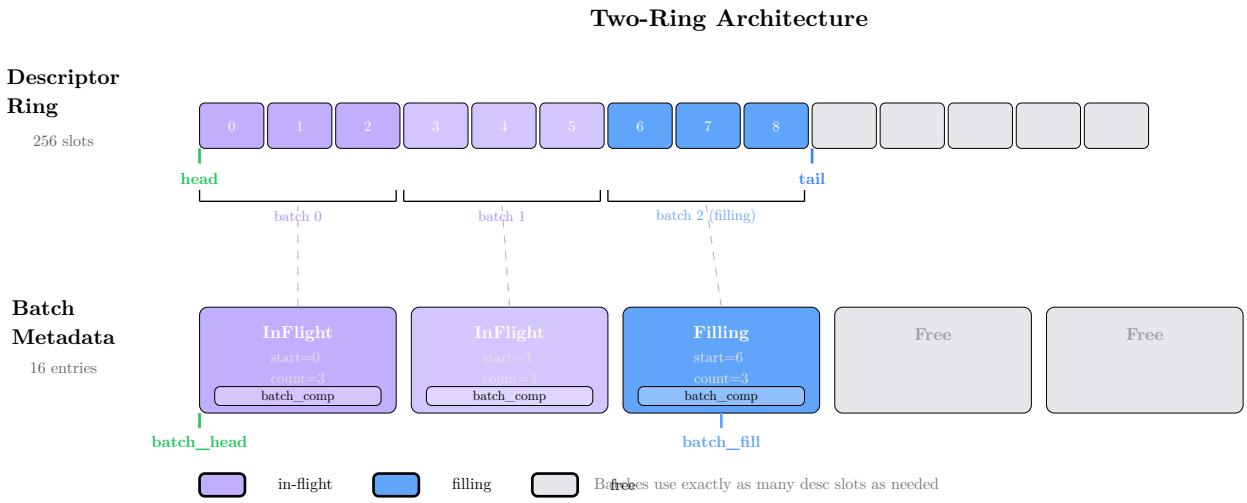

Figure 4: Ring-buffer architecture. Descriptor ring stores descriptors contiguously; batch metadata ring tracks start, count, and state. A batch of 3 uses exactly 3 slots, not 32.

How it works:

- `submit()` appends to the descriptor ring and the current Filling batch. When `count >= max_batch_size_`, the batch is auto-submitted.
- `poll()` seals and submits any partial Filling batch, then reclaims completed batches (advancing `desc_head_`).
- **Wrap-around:** if the next slot crosses the ring boundary, the current batch is sealed early and a new one starts at index 0 (ensures contiguity for DMA).

Advantages over double-buffered:

- **No blocking** – 16 batch slots vs. 2
- **Better utilization** – consistently full batches instead of partial ones from frequent `poll()`
- **No wasted space** – batches use exactly the descriptor slots they need

3.4.1 Wrap-Around

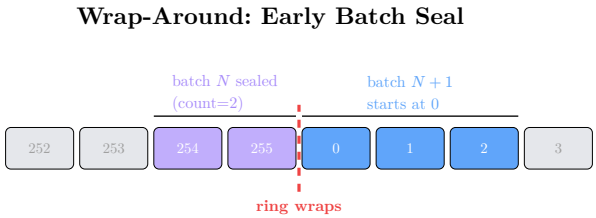

Figure 5: When the next descriptor would cross the ring boundary, the current batch is sealed early and a new batch starts at index 0.

3.5 Mirrored-Ring (`MirroredRingSubmitter`)

> **Idea:** Same two-ring architecture as `RingSubmitter` (shared descriptor ring + batch metadata ring) but the descriptor ring is backed by a `MirroredRing`: the same physical pages are mapped twice contiguously in virtual memory via `memfd_create` + two `MAP_FIXED` mappings. This eliminates the wrap-around check entirely — a batch that starts near the end of the ring sees contiguous memory via the mirror region.

How it works:

- Identical to `RingSubmitter` except: no wrap-around check in `submit_descriptor()`. When a batch spans the ring boundary (e.g. start=480, count=32 with ring size 512), slots 480..511 and the mirrored copies at virtual addresses 0..31 are contiguous in the process's virtual address space.
- Same `pre_poll()` / `drain()` / `reclaim_completed()` logic as `RingSubmitter`.
- `MirroredRing` RAII wrapper handles `memfd_create`, `ftruncate`, and double-mapping cleanup.

Advantages over `RingSubmitter`:

- **No early batch seals** — batches always fill to `max_batch_size_` (better utilization)
- **Simpler hot path** — no conditional branch for wrap-around
- **Same memory footprint** — identical ring sizes

Trade-offs:

- Requires `memfd_create` and `MAP_FIXED` (Linux-specific)
- Page-aligned allocation (ring size rounded up to page boundary)

4 Doorbell Cost Comparison

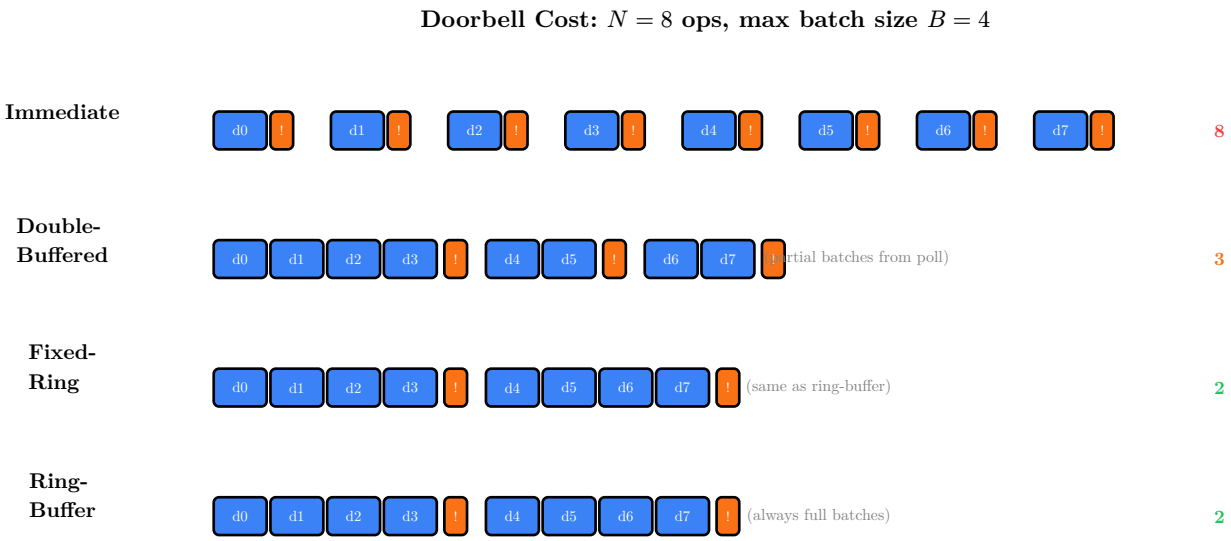

Figure 6: Doorbell count comparison for 8 operations. Immediate: 8. Double-buffered: 3 (partial batches from `poll()`). Fixed-ring and ring-buffer: 2 (always full batches of B).

5 Benchmark Results

All benchmarks: sliding window, inline polling, NoLock queue, concurrency 16, 8-byte messages, 75k ops/iter, 3 iterations. Platform: Intel 4th Gen Xeon Scalable, DSA configured via `accel-config`.

5.1 Message Rate

Operation	Double-Buffered (MPPS)	Ring-Buffer (MPPS)	Speedup
data_move	2.60	4.29	**1.65x**
mem_fill	4.13	8.41	**2.04x**
compare	4.61	8.09	**1.76x**
compare_value	4.69	8.08	**1.72x**
dualcast	4.62	7.99	**1.73x**
crc_gen	4.68	8.13	**1.74x**
copy_crc	2.36	2.96	**1.25x**
cache_flush	2.41	2.94	**1.22x**

Table 2: Message rate comparison at concurrency 16, 8-byte messages.

Largest gains on lightweight ops (mem_fill, compare) where software overhead dominates over hardware execution time.

5.2 Latency

Operation	Double-Buffered avg (ns)	Ring-Buffer avg (ns)	Double-Buffered p99 (ns)	Ring-Buffer p99 (ns)
data_move	4860	3247	6547	4955
mem_fill	3128	1678	3969	2155
compare	2806	1763	3615	2536
compare_value	2749	1765	3545	2527
dualcast	2806	1790	3612	2573
crc_gen	2759	1750	3553	2480
copy_crc	6129	5047	6964	7804
cache_flush	5973	5081	6745	7083

Table 3: Latency comparison at concurrency 16, 8-byte messages.

Average latency drops 30–46% for lightweight ops. The p99 for copy_crc and cache_flush shows slight regression with `RingSubmitter`, likely from occasional wrap-around batch splits. The `MirroredRingSubmitter` eliminates this by removing wrap-around handling entirely.

6 Performance Analysis

Three factors explain the 1.2–2.0x speedup:

1. **No submission blocking** — 16 batch slots vs. 2. Back-pressure is extremely rare.
2. **Better batch utilization** — auto-submit at `max_batch_size_` yields consistently full batches, whereas double-buffered submits partials on every `poll()`.
3. **No wasted descriptor space** — tightly packed ring vs. 2×32 fixed slots.

The fixed-ring ablation (Section 3.3) shares factors 1 and 2 but not 3: it also uses 16 in-flight batches with auto-submit, yet wastes descriptor space like double-buffered. Its performance closely matches ring-buffer, confirming that in-flight depth — not memory packing — is the dominant factor.

The mirrored-ring variant (Section 3.5) further improves on ring-buffer by eliminating early batch seals at the ring boundary, ensuring consistently full batches.

7 Strategy Comparison

Property	Immediate	Double-Buffered	Fixed-Ring	Ring-Buffer	Mirrored-Ring
Doorbells per N ops	N	$\lceil N/B \rceil$	$\lceil N/B \rceil$	$\lceil N/B \rceil$	$\lceil N/B \rceil$
Staging buffers	None	2 fixed arrays	16 fixed arrays	1 shared ring	1 mirrored ring
Max in-flight batches	N/A	2	16	16	16
Submit blocks?	No	Yes (on prev batch)	No	No	No
Descriptor waste	None	Up to 2×32 slots	Up to 16×32 slots	Only at wrap	None
Memory footprint	0	≈4 KB	≈34 KB	≈33 KB	≈33 KB
Wrap-around handling	N/A	N/A	N/A	Early batch seal	None (VM mirror)
Submitter class	`DirectSubmitter`	`StagingSubmitter`	`FixedRingSubmitter`	`RingSubmitter`	`MirroredRingSubmitter`

Table 4: Comparison of submission strategies. B = max batch size (typically 32).

8 Sizing

Parameter	Value	Rationale
`kDefaultNumBatches`	16	16 concurrent in-flight batches
`kDefaultBatchSize`	32	Default max descriptors per batch
`desc_ring_size_`	`round_up_pow2(32 #sym.times 16)` = 512	Dynamically computed; power-of-2 for masking
`max_batch_size_`	min(hw, 32)	Per-batch limit from hardware
Total memory	≈33 KB	512×64 B desc ring + $16 \times$ batch metadata

Table 5: Ring-buffer sizing parameters. Ring size is computed at `init()` time, not a compile-time constant.

Parameter	Value	Rationale
`kDefaultNumBatches`	16	Same depth as ring-buffer
`batch_capacity_`	32	Per-entry descriptor array size
`max_batch_size_`	min(hw, 32)	Per-batch limit from hardware
Total memory	≈34 KB	16 × (32 × 64 B + 32 B comp)

Table 6: Fixed-ring sizing parameters. Higher memory footprint than ring-buffer due to per-entry fixed arrays.

9 Implementation

9.1 Type Erasure and Composition

All five submitters satisfy the `DescriptorSubmitter` concept and are composed with `DsaEngine<Submitter, QueueTemplate>` to produce concrete DSA types. The `DsaFacade` interface (via Microsoft's proxy library) provides type erasure through `DsaProxy`, enabling runtime strategy selection. All batch submitters (`StagingSubmitter`, `FixedRingSubmitter`, `RingSubmitter`, `MirroredRingSubmitter`) use composition — each contains a `DirectSubmitter direct_` member for raw MMIO submission, rather than using inheritance.

9.2 Template Parameterization

`DsaEngine` is templated on both submitter type and queue type (`SingleThreadTaskQueue`, `MutexTaskQueue`, `TasSpinlockTaskQueue`, `SpinlockTaskQueue`, `BackoffSpinlockTaskQueue`, `LockFreeTaskQueue`, `IndexedSingleThreadTaskQueue`) with explicit instantiations in separate `.cpp` files (e.g. `dsa_staging_instantiate.cpp`, `dsa_ring_instantiate.cpp`, `dsa_mirrored_ring_instantiate.cpp`).

10 Future Work

- **Higher concurrency benchmarks** at 32–64 to validate ring depth sufficiency
- **Shared WQ contention** measurements – batching reduces `ENQCMD` retries on multi-tenant systems
- **Adaptive batch sizing** based on submission rate and hardware queue depth
- **Explicit batch sender** API: `dsa_batch(sender1, sender2, ...)` for application-level ordering